

An Iterative and Agile Process Model for Teaching Software Engineering

Maria Isabel Alfonso and Antonio Botía

Dept. of Computer Science and Artificial Intelligence. University of Alicante (Spain)
eli@dccia.ua.es, abotia@dccia.ua.es

Abstract

This paper describes the use of an iterative and agile process model in a software engineering undergraduate course. The proposed model serves both as an educational technique (for teachers) and as a subject of learning (for students). Details are given of the procedures and methods used and related management aspects are also addressed. The results obtained are discussed and compared with a previous experience using a weighted process model. This particular approach has been found a valuable and high satisfactory experience for both students and teachers.

1. Introduction

The software process model may be defined as a simplified description of a software process, presented from a particular perspective. Process models play a very important role in the development of large software engineering projects. Choosing the adequate model is an important managerial decision for the success of the project. The process model determines the sequence of tasks required to accomplish the project.

Similarly, in an educational context, the pedagogical methods used are one of the more important concerns for a teacher to fulfill the main objective: teaching. The pedagogical methods used describe the sequence of tasks for teaching the topics of any discipline. To a certain extent, we can establish an analogy between software manager-model processes, and teacher-teaching methods.

This analogy threw up two questions: (1) Can we use a software process model as a pedagogical tool to teach some of the main topics in software engineering? (2) What are the benefits of doing this? This paper tries to answer these questions. In answer to the first question, we propose the use of a mixed iterative and agile process model in a software engineering undergraduate course. In this paper, we discuss the proposed method from the teacher's standpoint, exposing the procedures and methods used, as well as related managerial aspects. In answer to the second question, we discuss the benefits of using the proposed method, mainly from the student standpoint, addressing the effective learning of the topics of the course.

In education, the discipline of Software Engineering is distinct from education in other disciplines, where designs and related products are not normally produced. Software Engineering learning is action-oriented: students must actively design and engineer software to be successful [1]. Accordingly, our approach subscribes to this idea and adds the following: "...and the teacher can act as a project manager in order to plan, monitor and control the learning process effectively".

The rest of paper is organized as follows: firstly, we present the Software Engineering course we have developed at the University of Alicante (Spain).

Secondly, we explain the proposed process model, we justify the reasons for this choice, and present the defined programme schedule, milestones and deliverables.

Thirdly, we analyse some of the organizational questions related to the proposed model, such as size, composition, organization, and student group cohesion.

We then discuss some of the pedagogical and managerial aspects of the proposed model, including the evaluation method used.

Next, we present an example project that we have been using during the 2003-04 academic year in order to illustrate the practical application of the model, and suggest other alternatives.

We show and analyse the results obtained comparing it with a previous experience using a weighted process model, and we finish by explaining some of the conclusions we have reached from our work.

2. Software Engineering at University of Alicante (Spain)

At the University of Alicante (<http://www.ua.es>), students can obtain a five-year graduate degree in computer science called *Computer Engineering*. The body of Software Engineering knowledge is mainly obtained from three 60-hour courses. In the first course, students learn about requirement specifications, and different methods used to analyse software engineering projects. The second is mainly focused on the study of software project design and the application of several design methods. The third and last course is the one dealt with in this paper. In this course, the aim is to complete students' knowledge of Software Engineering and practice group working.

We divide the course into 30 hours of theory and 30 hours of practice. In the theory classes, we summarize the main concepts of the following topics: process models, planning activities, testing process and configuration management process. In the practical classes, we apply the theoretical concepts explained by developing a project in working groups following the process model proposed in this paper.

The course is taught over four hours per week: two theory hours and two practical hours, spread over 15 weeks.

The course website can be visited at: <http://dccia.ua.es/dccia/inf/asignaturas/IS2>. The site contains the slides used in the theory room (adapted from Sommerville book [6]), and all information related with the work to be carried out in laboratory hours to accomplish the project.

3. The proposed iterative and agile process model

Iterative process models are primarily concerned with the idea of building software through the sequential development of several mini-projects known as *iterations*. An example of the iterative process is the so-called *Unified Process* (UP) or the variation, referred to as *RUP* (Rational Unified Process) [7] based in UML (Unified Model Language) [8]. More specifically, RUP is a project framework that describes a class of processes that are iterative and incremental. RUP-compliant process deliver functionality in small increments, each building on the previous, and each being driven by use cases rather than being the construction of a subsystem. Iterative process models pre-date agile process models.

Agile processes are low-overhead processes that accept that software is difficult to control (they are adaptative rather than predictive). They minimize risk by ensuring that software engineers focus on smaller units of work (they practice iterative development). They are distinguished from "heavier" processes thanks to their emphasis on values and principles rather than of processes (they are people-oriented rather than process-oriented) [9]. Examples of agile processes are XP and Scrum. XP (Extreme Programming) [10] is characterized mainly by continuous automated unit testing, pair programming, refactoring, shared code

ownership, simplicity and feedback. Scrum is characterized mainly by the development of named *sprints* (similar to iterations), and the use of a list of tasks named *backlog*, which drives team activity [11].

In as much as RUP is a project framework, it can accommodate a wide variety of processes. As a result it can be used in a very traditional waterfall style (heavyweight manner) or in an agile manner. Craig Larman is a strong proponent of using RUP in an agile manner [12]. We have decided to adopt a particular instance of agile RUP, as we adopt certain XP features (such as pair programming, shared code ownership, simplicity and feedback), and certain Scrum-based ideas, as we explain later. The main reasons for this decision are: (a) iterative process model are a good choice for carrying out successful large software projects, and RUP is the most widely used iterative process model; (b) RUP can be used in an agile manner to illustrate the practice of these modern and widely used methods; and (c) we have found several pedagogical advantages in the use of this for teachers, as we discuss in sections 5 and 7.

Figure 1 resumes the programme schedule that applies the proposed process model. It shows the milestones set and the deliverables generated. As in RUP, the iterations are grouped into four phases called *inception*, *elaboration*, *construction* and *transition*. Each phase is formed by one or several iterations of a fixed duration of two weeks (which corresponds to four laboratory hours, except for last iteration, which lasts three weeks). At the end of the iterations (which corresponds to set milestones) a partially complete system that is stable, integrated and tested is delivered, along with other artifacts¹.

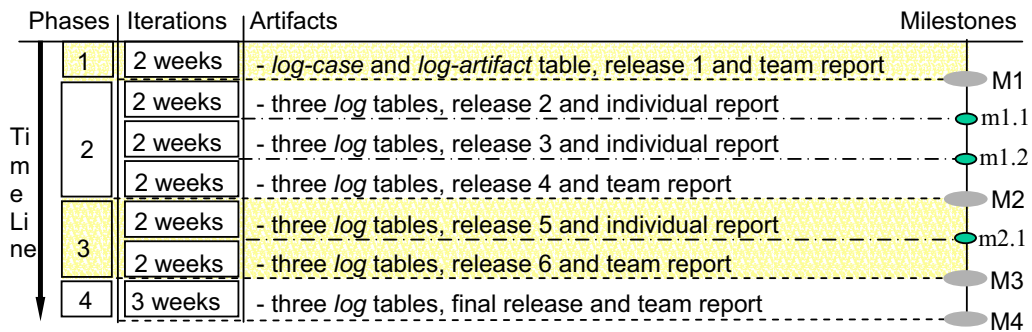


Figure 1. Schedule of process model proposed

There are four major milestones defined at the end of each phase: M1...M4, in which the teacher scores the work carried out by the team. Three minor milestones are also defined: m1.1, m1.2, and m2.1, in which individual work is scored.

During the inception phase teams are formed, well-defined roles are assigned, the infrastructure is put in place and the communications channels are established. A high level plan is defined by students, in which main use cases and other tasks are identified and assigned to the iterations of the project (we have called this artifact *log-case table*). At the end of this phase, the first major milestone is reached and the team members have a clear vision of the project, the available resources, and the customer's (teacher's) expectations.

During the elaboration phase, the architectural foundation of the software is established and high risks are eliminated. During the construction phase, we expect the use case descriptions to stabilize to a certain extent. Finally, during transition phase, the team is mainly taken up with deployment activities.

¹ RUP defines the term *artifact* to denote a discrete and cohesive collection of information generated during the development process.

During the elaboration phase and subsequent phases, teams work on the use cases and tasks planned in the *log-case table*. The items of this table are detailed and refined during iterations, and are modified if necessary. The log-case table drives team activity, in a similar way to the aforementioned *backlog* in Scrum.

The artifacts produced in each iteration are recorded in a *log-artifact table*. The artifacts produced are uml diagrams (use case, scenarios, classes and deployment), source code, executable code, ant files, test-case tables, and JUnit files, and they are delivered as part of each release in a directory named *repository*.

And we also use a so-called *log-trace table* to record the state of each use case (an entry of the log-case table) during each iteration, this state can be: C (completed), N (not initiated), or P (partially completed). This table constitutes a trace of the work carried out by each team during the course.

Finally, each team generates two report types: an individual report (at the end of each iteration); and a team report (at the end of each phase). The individual report contains the following information: (a) use cases worked on and current state of said use cases; (b) artifacts produced; (c) role played (see section 4); and (d) responsibility of the pair (see section 4). The team report is prepared by the same pair that is responsible for the repository. This is a brief document with three items: (a) report on the work carried out by the team during the phase; (b) comparison of objectives planned at the beginning of the phase, with objectives achieved at the end of the phase; and (c) report on the reasons for the success or failure in achieving the objectives, indicating the problems that occurred and how (and if) they were solved. The team report is also explained verbally to the teacher by the corresponding pair during a maximum time of five minutes.

4. Organizational aspects

Each team is made up of six students. The students form the teams themselves, except in particular cases, e.g. when there are students that like to join a team with already has six members. In this case, the teacher moves these students into other groups. As a result, there will be groups with students that may have incompatible personalities and this will probably lead to team communication and coordination problems.

Each team is organized into three pairs, called A, B and C (see Figure 2). The A, B, and C pairs are responsible for log-case, log-artifact, and log-trace tables, respectively. Each member of a pair plays one of two roles: programmer or tester. The programmer generates the source code and the class and deployment diagrams. The tester generates the use cases and scenarios, and the test cases, and controls the quality of the source code. The three pairs A, B, C, rotate between team members every two iterations. Moreover, in each phase, one of the pairs is responsible for the repository and team report. The communication is horizontal between the pairs, and between the members of a pair. The teacher communicates with all

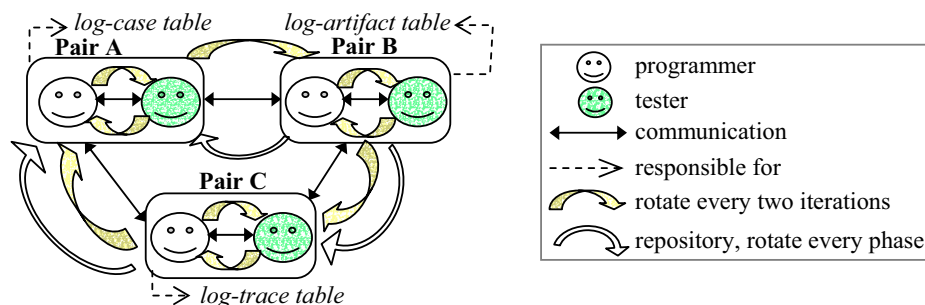


Figure 2. Organizational model proposed

members of the group.

Each team assigns the work freely to each pair, with the condition that the workload of each pair must be approximately the same. This is supervised by the teacher in each iteration, where the work to be assigned is driven by the log-case table and the log-trace table from the previous iteration.

Each laboratory work session is organized into three periods: (a) put together (half an hour); (b) pair work (an hour); and (c) inspection (half an hour). During the first period each pair must combine their work with the others and update both the *repository* and the three *log* tables. During the second period, each pair works in tasks corresponding to the current iteration. During the third period, the work corresponding to periods (a) and (b) are inspected by the teacher as is explained in the following section.

5. Pedagogical and managerial issues

Our approach is group working centered and software project-based. This is not new: other approaches use group work to teach software engineering, such as [2], [3], [4], [5]. In our case, we propose a particular software process model that guides the work of the groups and emphasizes the pedagogical aspects (issues). Consequently, the proposed process model has been designed with both the teacher (pedagogical aspects) and the student (active learning of modern software engineering techniques) in mind.

The teacher acts as a manager for each team (somewhat similar to Scrum master), as he/she guides, monitors and controls the work carried out by the students.

To guide the work carried out by students effectively, the teacher provides them with: (a) a document explaining the model process to be applied, organizational issues, the project to be carried out, the tools to be used, and the monitoring and control processes; (b) a complete set of templates for all the artifacts to be delivered, including log-tables, individual and team reports and test-case tables. (in order to reduce the overhead of documenting the project, all templates are simple calc sheets designed to contain useful information; students can complete their calc sheets in a short period of time); and (c) an initial release project, (the so-called zero release), which contains an initial repository suggesting a proposed architecture and some of the code details to reduce programming effort required of the teams.

To monitor the work carried out by the students effectively, the teacher inspects and scores the deliveries at each defined milestone. This inspection takes place during the third period of the corresponding laboratory session. In order to reduce the inspection time, the teacher has a template (also in the form of a simple calc sheet) for collecting the teams' scores. The scores on the template reflect both the individual work and team work, together with other related items such as team coordination, communication and cohesion. Each individual has two scores: one obtained from their individual work, and one obtained from the team. The final individual score is the mean value of the scores obtained at each milestone.

To control the work carried out by the students effectively, the teacher informs the teams of the score obtained, enabling feedback with students regarding the errors made. This favours student learning in that they can correct their errors in subsequent iterations. Each iteration is developed by a task sequence called *disciplines*. The disciplines are executed concurrently throughout the lifetime of the project. As the project matures, the emphasis on certain disciplines increases or decreases, e.g. little code is written early in the inception phase, and little design work is carried out in the transition phase. However, disciplines can be executed, and indeed are executed, at any time during the project.

The management tasks of guiding, monitoring and controlling the development of the project outlined above corresponds to the pedagogical knowledge-providing tasks, the evaluation of the knowledge acquired, and the driving of the learning process.

The student benefits from the proposed model are as follows: (1) the tasks carried out by the teacher serve as a practical example of how to manage a software team project; (2) they gradually learn the difficult task of group working as a consequence of model phases, where the inception phase requires more communication with the teacher (who also acts as the customer), and the subsequent phases gradually require less communication with the teacher and more with team members, so, the tasks move from building consensus with the teacher to building consensus within the team; (3) students learn by the use of repetitive tasks during iterations; and (4) the model proposed is an example of the application of modern process models.

Finally, students benefit from the tools used to develop the project. We use Eclipse 3.0 (<http://www.eclipse.com>) with the Omondo plug-in (<http://www.omondo.com>). These are modern tools that generate and synchronize design (UML diagrams) and programming (java language), automatically generate code for testing (using JUnit), build the project effectively (using Ant), and effectively manage the configuration elements generated (using CVS).

6. Example project

The project proposed during the academic year 2003-04 consists of developing a tool to automatically monitor the schedule of a set of tasks (which we named *workflow*). The project can be divided into three modules (each one can be assigned to one pair in a team): (1) the client module, (2) the server module, and (3) the monitor module.

The client module includes a layer presentation in which a super-user can define the workflows, the human resources assigned to the tasks, the duration of the tasks, and the start date of the workflows. The human resources assigned to a workflow act as normal system users and communicate the completion of the assigned tasks to the server module. Normal users can also consult workflow data, e.g. tasks completed or pending.

The server module stores the data validated in the client module into a database (we have used MySQL) to be monitored by the monitor module. The server module communicates with the monitor module to activate workflow monitoring.

The monitor module processes several workflows simultaneously. Once a workflow is activated, the monitor module regularly checks the completion of the corresponding tasks and sends a mail to the assigned users if the tasks are not completed in time.

A more detailed description of the project is delivered to students as part of the course material. This material is currently in Spanish only, and is to be translated into English in the future.

We have chosen this project because it helps students learn and practice other software engineering issues, such as project scheduling, by having to generate the activity chart associated with the workflow. Accordingly, we think that this type of project is a good choice, as it reinforces students' knowledge of software engineering. Other project examples we have used in past courses include a tool that derives tests cases, a tool that generates model process templates, a tool that implements COCOMO estimation method, and a tool that generates project schedules.

7. Discussion of results

During the academic year 2003-04, the number of students has been 153. They are divided into 22 groups of six, and 3 groups of seven. In one laboratory session (two hours per week) there are a maximum of three or four teams. We are five teachers, and we have worked in team software development since 1995. In past years, we have proposed several process models for the effective learning of software engineering [13], [14]. More specifically, in [14]

we used a weighted UP-based process model driven by functional services instead of use cases, and worked with groups of five students. The new approach we propose emphasizes the role of the process model as a pedagogical tool for the teacher. As result, we have obtained a greater level of satisfaction in both teachers and students.

Table 1 shows a comparison of the results of the year 2003-04 with some of the results presented in [14] (obtained during the last two years).

Course	Team	Students		Teachers	
		Success	Extra time	Scoring effort	Activity lab
2001-02	5	22 of 28	≤2h	≤2h	65%
2002-03		26 of 28			
2003-04	6	23 of 25	≤1h	≤1h	85%

Table 1. Some of the results obtained

This year, 23 groups from a total of 25 have developed a successful project; this means that only two groups have not completed the project. We see that a similar result was obtained in 2002-03. The improvement of 2002-03 result in comparison with 2001-02 is basically the result of the teachers' greater experience, in that they also learn from the difficulties encountered by students and can consequently guide the students' work more effectively. We also expect this percentage to increase for 2004-05.

As for the time spent by students on the proposed project, the groups needed to work at home for a maximum of two extra hours per week in order to complete the project during 2001-03. In 2003-04, the time spent by students was reduced by 50%. This is a consequence of the agile character of the new proposed process model, which reduces the overload of documenting the project. We have reduced the volume of documents to be delivered to a set of simple, standard calc sheets, and a set of automatically generated UML diagrams. This means that students can document the project effectively in a short time with all the information required for the future maintenance of the system produced.

Teachers also benefit from the proposed process model thanks to a 50% reduction in the efforts for scoring students. This effort is reduced in several ways: (a) we have standardized and simplified the set of artifacts and the repository students have to deliver in each iteration, providing the corresponding templates in such a way that the teacher knows that all the documents (including the code) will have the same structure and format, and the same items of information; (b) the teacher reviews the deliveries with the students during laboratory hours and receives a verbal report at each phase with three points that summarise the work carried out by the groups. This verbal report is very important and improves students' capacity for expression, concision and clarity in that they have a time limited to a maximum of five minutes to summarise several weeks' group work; and (c) the teacher can execute the partially implemented system at each iteration (every two weeks), and can compare the code with the documents delivered with ease.

Finally, teachers benefit from short iterations, development that is driven by use cases, and the development of high-risk use cases in early iterations. These risks are addressed up front and the teacher can help solve problems actively during laboratory hours. This increases the teacher's work in laboratory hours and reduces his/her total work in answering students' questions. During the academic year 2003-04 the activity carried out by teachers during laboratory hours represents 85% of the total time spent, reducing the extra hours of the total time spent during 2001-03 by 20%.

8. Concluding remarks

In this paper, we have presented a particular iterative and agile process model that serves both as a pedagogical tool for the teacher, and as a subject matter for students. We have been motivated by our idea of benefiting the subject under study (*what* we are learning) as a method for teaching it (*how* we are learning).

We have analyzed the proposed model from the teacher's standpoint and discussed the main advantages of this approach: (1) it provides knowledge efficiently, (2) it evaluates the knowledge acquired gradually, and (3) it drives the learning process effectively. From the students' standpoint, we also demonstrate the main benefits: (1) learning managerial development issues from the teaching methods used, (2) learning development issues concerning widely used modern process models, and (3) learning group working gradually.

The results we present have been compared with a previous experience using another weighted approach. We have used the data corresponding to three academic years. The preliminary data obtained reflects a more satisfactory experience for both teachers and students. We intend to apply and improve this process model in the future.

10. References

- [1] M. Murphy, R. Halstead-Nussloch, "Learning and Instructional Issues in Software Engineering", Workshop: Beg, Borrow or Steal: Using Multi-disciplinary Approaches in Empirical Software Engineering Research (ICSE'2000), Limerick, Ireland, June 4-11th, 2000.
- [2] A. J. Cowling, "The Crossover Project as an Introduction to Software Engineering", Proceedings of the 17th Conference on Software Engineering Education and Training (CSEET'04). IEEE CS Press, Norfolk, Virginia, March 01-03, 2004, pp. 12-17.
- [3] A. Seffah, P. Grogono, "Learner-Centered Software Engineering Education: From Resources to Skills and Pedagogical Patterns", Proceedings of the 15th Conference on Software Engineering Education and Training (CSEET'02). IEEE CS Press, Covington, Kentucky, February 25 - 27, 2002, pp. 14-23.
- [4] D. P. Groth, E. L. Robertson, "It's All about Process: Project Oriented Teaching of Software Engineering", Proceedings of the 14th Conference on Software Engineering Education and Training (CSEET'01). IEEE CS Press, Charlotte, North Carolina, February 19 - 21, 2001, pp. 7-17.
- [5] B. Boehm, D. Port, D. Klappholz, Workshop on Tailoring a Successful Project-Based Course, Conference on Software Engineering Education and Training (CSEET'03), Madrid, Spain, Mars 20-22, 2003.
- [6] I. Sommerville, "Software Engineering", 6th edition, Addison Wesley, 2002.
- [7] I. Jacobson, G. Booch, J. Rumbaugh, "The Unified Software Development Process", Addison-Wesley, 1999.
- [8] Object Management Group, "OMG Unified Modeling Language Specification", 1998. <http://www.omg.org>.
- [9] M. Fowler, "The New Methodology", April, 2003, <http://www.martinfowler.com/articles/newMethodology.html>
- [10] K. Beck, "Extreme Programming Explained: Embrace Change", Addison-Wesley, 1999.
- [11] M. Beedle et al., "SCRUM: An extension Pattern Language for Hyperproductive Software Development", Pattern Languages of Program Design 4, Addison-Wesley, 2000, pp. 637-651.
- [12] C. Larman, "Applying UML and Patterns: An introduction to Object-Oriented analysis and design and the Unified Process", 2nd edition, Prentice Hall. 2002.
- [13] M.I. Alfonso, F. Mora, F. Martínez, "La programación en grupos de trabajo", JENUI'98, Andorra, July 9-10, 1998, pp. 181-185.
- [14] M.I. Alfonso, F. Mora, "Learning Software Engineering with Group Work", Proceedings of the 16th Conference on Software Engineering Education and Training (CSEET'03). IEEE CS Press, Madrid, Spain, Mars 20-22, 2003, pp. 309-316.